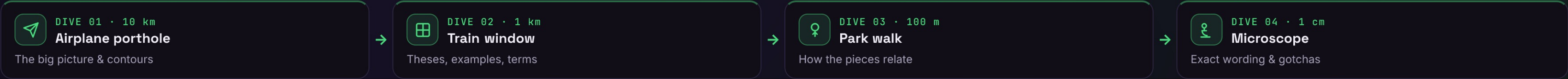




Strategy

Define a family of interchangeable algorithms and let the chosen one vary independently from the client that uses it.



 Airplane porthole

DIVE 01 · ORIENTATION

▼ 10 km

Encapsulates each algorithm behind one interface — so the client can swap implementations without touching its own code.

— THE CONTOURS

• One interface, many algorithms
— each a separate, swappable class

• Composition over inheritance
behavior is injected, not subclassed in

• Client chooses
— it sets the strategy; the strategy never self-swaps

— USE IT WHEN — GoF applicability

 Many variants of one task
differ only by algorithm and should be selectable at runtime

 A class hides conditional behavior
in branches that each really want to be their own algorithm

 Behavioral family: Strategy · State · Template Method · Command — Strategy swaps a whole algorithm via composition, where Template Method varies steps via inheritance.

↓ DIVE DEEPER

 Train window

DIVE 02 · KEY OBJECTS

▼ 1 km

— THE THREE PARTS

Strategy
the common interface every algorithm implements

ConcreteStrategy
one self-contained algorithm behind that interface

Context
holds a Strategy and delegates the work to it

— IN THE WILD

 Array sort comparator
sort((a,b) => ...) is a strategy passed as a function

 Passport.js
pluggable auth strategies (local, OAuth, JWT)

 java.util.Comparator
interchangeable ordering injected into sort

 Compression codecs
gzip · brotli · deflate behind one stream API

 Context configured with a Strategy → Context.doWork() forwards to strategy.execute(data) → result returns.

GoF intent: “Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.”

↓ DIVE DEEPER

 Park walk

DIVE 03 · CONNECTIONS

▼ 100 m

Client chooses
setStrategy(s)

→

Context holds it
composes the algorithm

→

doWork() delegates
strategy.execute(data)

→

✓ Algorithm runs
result returned to client

— WHY IT FOLLOWS

→ One interface => algorithms are interchangeable at the same call site

→ Composition => swap behavior at runtime without subclassing the Context

→ Client-set => the caller decides which algorithm; nothing self-transitions

🔗 Often a stateless Flyweight shared across contexts · a Factory may select the strategy · same UML as State, but Strategy is set by the client and never self-transitions.

— STRUCTURE · UML

Context

- strategy: Strategy

+ setStrategy(s: Strategy): void

+ doWork(): Result

Context delegates doWork() to its current Strategy; the client picks the algorithm via setStrategy().

— WHAT IT BUYS YOU

✓ Family of algorithms — a clean class hierarchy of reusable, related behaviors

✓ Replaces conditionals — no sprawling if/switch selecting behavior inline

✓ Choice of implementations — same task, different space/time trade-offs, picked at runtime

✓ Client must know strategies — the caller has to understand the options to choose well

✓ Communication overhead — the interface may pass data a given strategy never uses

↓ DIVE DEEPER

 Microscope

DIVE 04 · DETAILS

▼ 1 cm

Definition: “Define a family of algorithms, encapsulate each one, and make them interchangeable.”
— GoF, “Design Patterns”, 1994

— THE CODE

```
interface Strategy {
  execute(d: number[]): number[];
}
const asc: Strategy = { execute: (d) => d.sort() };
class Sorter {
  constructor(private s: Strategy) {}
  run(d: number[]) { return this.s.execute(d); }
}
new Sorter(asc).run([3, 1, 2]); // client picks
nums.sort((a, b) => a - b); // or just a function
```

— COMPARE THE ALTERNATIVES

Property	Strategy	State	Template Method	if/else
Client picks behavior	✓	✗	✗	✓
Swaps own transition	✗	✓	✗	✗
Composition (not inherit)	✓	✓	✗	~
Algorithms interchangeable	✓	~	~	✗
Open to new variants	✓	✓	~	✗

Best fit → Strategy: client-chosen algorithm · State: self-driven modes · Template Method: fixed skeleton, varied steps · if/else: one-off branch. (“~” = partial.)

— INTERVIEW PITFALLS

Q “Strategy vs State?”
Identical UML; a Strategy is chosen by the client and stays put, State objects swap themselves as internal conditions change.

Q “Do I need a class per strategy?”
No — in JS a function is a perfectly good strategy; use classes when an algorithm carries its own state or configuration.

Q “Strategy vs Template Method?”
Strategy varies a whole algorithm via composition and runtime swapping; Template Method varies steps via subclass inheritance.

Q “Are strategies stateless?”
Usually yes, which makes them safe to share as singletons; if one holds state, do not reuse the instance across contexts.

— VARIANTS

Class strategy
one class per algorithm — classic GoF, fully typed

Registry / factory
look up a strategy by key from a map or factory

Null-object
a no-op strategy avoids null checks in Context

Function / lambda
pass a plain function in JS — no class needed

Default strategy
Context ships a sensible algorithm if none is set

Parameterized
configure one strategy via constructor args

— NUANCES & GOTCHAS

• Usually stateless — algorithms hold no per-call state, so one instance can be shared everywhere

• Client must choose — the caller needs enough context to pick the right strategy

• Function vs class — in JS a closure is often plenty; classes earn their keep with state or config

• vs Template Method — Strategy varies the whole algorithm by composition; Template Method varies steps by inheritance

• vs State — same UML, but Strategy is client-set and never self-transitions between algorithms

• Data plumbing — passing args to execute() vs holding them risks a fat interface or hidden coupling

— WATCH OUT

 **Anti-pattern:** If there is only one algorithm, or variants never change at runtime, a single function or a simple conditional is lighter than a whole interface and class hierarchy.

 **Modern take:** In modern JS/TS a strategy is often just a first-class function — pass the comparator or handler directly and skip the ceremony of an interface plus classes.

• Vasyi Krupka · Senior Fullstack Engineer · learn-by-diving series

Source: GoF “Design Patterns” (1994) **v1**